MyoWare 2.0 & Wireless Shield — Hardware Reference

---

0. EMG Sensor Comparison: MyoWare 2.0 vs SEED/Grove EMG

Why RAW signals from both sensors look similar

Both sensors output a **bandpass-filtered, amplified EMG signal** centered around a DC bias — not truly "raw" ADC from bare electrodes. The oscillating waveform (fast bursts during contraction, near-flat at rest) is the same underlying physiology captured by the same class of analog front-end.

Feature	MyoWare 2.0	SEED/Grove EMG
Instrumentation amplifier	✅ Medical-grade INA	✅ Consumer-grade
Hardware bandpass	**20–500 Hz**	~20–500 Hz
Hardware envelope output	✅ **ENV pin** (rectified + smoothed)	❌ None
DC bias	~1843 counts (stable)	Less stable
Electrode connectors	Snap (standard Ag/AgCl)	Integrated, less flexible
SNR	Higher	Lower

MyoWare "RAW" = output of the hardware bandpass filter, *before* the hardware envelope detector — not raw ADC from bare electrodes.

SEED EMG = same class of filtered signal, but no hardware envelope output.

Which is better for fatigue detection (MDF-based)?

MyoWare 2.0 is the better choice. Reasons:

1. **Hardware ENV pin** is unique to MyoWare 2.0 — SEED has no hardware envelope output. Both sensors share a similar ~20 Hz hardware low cutoff, but MyoWare's ENV pin provides a free signal quality / validity indicator that SEED cannot match.
2. **Higher SNR** from the medical-grade instrumentation amplifier gives cleaner FFT spectra, which is critical for reliable MDF shift detection across a tennis session. The `sensordata_plotter.py` ENV comparison subplot uses the ENV pin to detect saturation and electrode contact loss in real time. SEED requires software-only envelope.
3. **Software 50 Hz high-pass** in `preprocess_emg()` further rejects the 20–50 Hz band (residual motion artifact + low-frequency EMG components less useful for MDF analysis). This is a software choice, not a hardware difference — both sensors have a ~20 Hz hardware low cutoff.

4. **Existing codebase is built around MyoWare** — `btCentral.py`, `FatigueMonitor`, the Android app all assume dual-output (RAW + ENV). Switching to SEED would require significant rework with no benefit.

Bottom line: MyoWare's hardware bandpass + ENV pin + higher SNR make it the right choice for MDF-based fatigue detection. The SEED sensor is adequate for simple amplitude-based detection but not for frequency-domain fatigue analysis.

---

1. Background (RAW Signal Wiring Fix)

The Fatigue Monitoring system relies on capturing raw electromyography (EMG) signals from a SparkFun MyoWare 2.0 Muscle Sensor connected to a Wireless Shield (ESP32). To track muscle fatigue, the offline analysis script (`emg_util.py`) computes the Median Frequency (MDF) and Mean Frequency (MNF) of the muscle's electrical firing using Fast Fourier Transforms (FFT).

For frequency analysis to work, **a clean, high-frequency, alternating-current (AC) `RAW` signal is strictly required**. The rectified and integrated Envelope (`ENV`) signal provided by the sensor cannot be used for frequency-based fatigue tracking because the hardware low-pass filter destroys the necessary high-frequency data.

2. The Problem: Data Log Analysis

During testing, the captured EMG CSV logs showed clear signs of hardware disconnect on the RAW channel.

Log Batch 1:

```
dev_us,emg_seq,env,raw,ref,frame_seq,host_rx_ms
230836391,0,1751,65,0,0,1770606129424
230836891,1,1746,128,0,0,1770606129424
230837391,2,1730,173,0,0,1770606129424
```

- **Observation:** The `raw` ADC values are extremely low (65 - 173). On an ESP32 12-bit ADC (range 0-4095), an AC-coupled resting RAW signal from the MyoWare sensor should hover near the midpoint (roughly `2048`, representing ~1.65V).

Log Batch 2 (During Exertion):

```
dev_us,emg_seq,env,raw,ref,frame_seq,host_rx_ms
28487653,42,3824,1011,0,4,1770609737530
28488153,43,3824,1009,0,4,1770609737530
28488653,44,3824,1005,0,4,1770609737530
```

- **Observation:** The `env` (Envelope) is registering massive exertion, pegged near the ADC maximum at `3824`. However, the `raw` signal is completely dead, flatlining at exactly `1005` to `1011` (a variance of only ±4 units).
- **Conclusion:** It is physically impossible for the sensor to output a massive Envelope voltage without a correspondingly massive, wildly oscillating RAW AC waveform. The static `raw` column proves that

the ESP32's analog pin is completely disconnected from the sensor's RAW output and is simply floating, picking up static background capacitance.

3. Root Cause (Pogo Pin Failure)

The SparkFun MyoWare 2.0 ecosystem uses snap connectors to stack boards. However, **the snap connectors themselves only route three signals**: `VIN`, `GND`, and `ENV`.

To pass the `RAW` and `REF` signals to the Wireless Shield, SparkFun designed the shield with two tiny, spring-loaded **pogo pins** on its underside. These pogo pins are meant to physically press against the `RAW` and `REF` plated through-hole (PTH) pads on the Muscle Sensor board below it.

If your CSV data shows a floating `RAW` pin (as seen in the logs above), it means **the pogo pin connection has failed**. Common reasons for this include:

1. **Misalignment:** The boards are not snapped together perfectly flat, preventing the pogo pin from touching the pad.
2. **Damaged Pin:** A pogo pin is stuck in the compressed position, bent, or missing.
3. **Dirty Contact:** Flux residue, oxidation, or skin oils on the Muscle Sensor's `RAW` PTH pad are preventing electrical conductivity.

Because `emg_util.py` ignores the hardware `ENV` column and relies entirely on the `RAW` column, the offline fatigue pipeline will fail completely when this pogo pin connection is compromised.

4. The Fix

Step 1: Inspect and Clean (Non-Destructive)

1. Unsnap the Wireless Shield from the Muscle Sensor.
2. Look at the bottom of the Wireless Shield. Verify the two gold pogo pins are present and springy.
3. Clean the `RAW` PTH pad on the top of the Muscle Sensor with rubbing alcohol.
4. Reseat the boards, ensuring they are perfectly parallel and pressed tightly together.
5. Capture a new log to see if the `raw` column now hovers near `2048` at rest.

Step 2: The Hardwire Bypass (If Pogo Pins Fail)

If the pogo pins are damaged or unreliable during active sports/movement, you must bypass them with a hardwired connection:

1. Locate the **RAW** PTH pad on the MyoWare Muscle Sensor board.
2. Locate the pad labeled **`32`** on the underside of the Wireless Shield.
3. **Solder a flexible jumper wire** directly between these two pads.

The "A4" Naming Trap

The original firmware code comments incorrectly assumed that the pad labeled "A4" on the shield routed to ESP32 GPIO 32:

```
// ==================== EMG Pins ====================
static constexpr int PIN_ENV = 39; // A3 (ADC1)
static constexpr int PIN_RAW = 32; // A4 (ADC1) - THIS COMMENT WAS WRONG!
```

On original Arduino Uno boards, A4 is the I2C SDA pin. To maintain compatibility with Uno shields, SparkFun routed the ESP32's actual I2C SDA pin (**GPIO 21**) to the hole labeled "A4". GPIO 21 is strictly a digital pin and has no ADC capabilities.

Because the code was listening to GPIO 32, but the physical wire was soldered to the digital I2C bus (A4), the ESP32 ADC was reading a floating pin.

How We Debugged It (The "Finger Touch" Test)

To prove the physical disconnect, a tiny diagnostic Arduino sketch was flashed to the ESP32 to print the values of all ADC1 channels to the Serial Monitor:

```
void setup() { Serial.begin(115200); }
void loop() {
  Serial.print("Pin36:"); Serial.print(analogRead(36)); Serial.print("\t");
  Serial.print("Pin37:"); Serial.print(analogRead(37)); Serial.print("\t");
  Serial.print("Pin39(ENV):"); Serial.print(analogRead(39)); Serial.print("\t");
  Serial.print("Pin32:"); Serial.print(analogRead(32)); Serial.print("\t");
  Serial.print("Pin33:"); Serial.print(analogRead(33)); Serial.println();
  delay(100);
}
```

While running this script, the user physically touched the "A4" pad with their bare finger. Because the human body acts as an antenna for 60Hz mains noise, touching an active ADC pin causes the readings to swing wildly (0 to 4095). **Touching A4 produced no change**, proving it was not connected to any of the ADC pins being scanned. Touching the pad explicitly labeled `32` on the back of the shield caused massive fluctuations, confirming the correct routing.

The "Ghost" Signal & ADC Crosstalk

During the period where the RAW pin was floating, the CSV logs showed the `raw` column maintaining a constant ~37% to ~44% ratio relative to the `env` column, with a correlation of 0.998.

Why does RAW mirror ENV if it is disconnected? This is caused by **ADC Multiplexer Charge Sharing (Crosstalk)**. The ESP32 ADC Continuous (DMA) mode switches channels at 40 kHz. The internal Sample-and-Hold (S/H) capacitor charges to the actively driven `ENV` voltage (GPIO 39). When the multiplexer switches to the floating `RAW` pin (GPIO 32), the pin has no driving source. The charge is distributed between the S/H capacitor and the parasitic capacitance of the floating pin/trace, resulting in a predictable fractional voltage drop (the ~40% ghost signal).

Because this ghost signal is just a slow DC-like trace, when the Python script passes it through `preprocess_emg` (which applies a high-pass filter to block DC noise), the ghost signal gets completely wiped out, resulting in a flatline.

Final Verification

After moving the solder wire to the correct pad (`32`), the `raw` signal finally showed true, independent, high-frequency AC oscillation.

To visualize this properly in the real-time Python plotter, a `--raw-zoom` flag was added to `sensordata_plotter.py`. The MyoWare sensor centers the resting RAW signal at ~1.65V (1800-2048 counts). The `--raw-zoom` flag dynamically zooms the Y-axis to perfectly track the baseline noise, allowing the user to clearly see the high-frequency bursts of muscle noise bulging outward exactly in sync with the hardware envelope.

Further down, the firmware configures the hardware Direct Memory Access (DMA) to capture high-speed data from `ADC1 channel 4` (which maps to Pin 32):

```
// PIN_ENV=39 -> ADC1 channel 3, PIN_RAW=32 -> ADC1 channel 4
static constexpr adc_channel_t ADC_CH_ENV  = ADC_CHANNEL_3;
static constexpr adc_channel_t ADC_CH_RAW  = ADC_CHANNEL_4;
```

Finally, inside the `emgDrainTask`, the data from channel 4 is saved into the `s.raw` variable and transmitted via BLE. Because the ESP32 is exclusively listening to `A4` for the raw signal, the jumper wire *must* go to this exact pad.

What about the REF pogo pin?

While the MyoWare Advanced Guide notes that monitoring the Reference (`REF`) electrode signal can be useful for advanced diagnostics (such as measuring common-mode noise on the user's skin), **you do not need to solder a bypass wire for the REF pin** with your current setup.

The proof is in the `myoware_imu_v2.ino` firmware. The code explicitly disables the REF pin:

```
static constexpr bool REF_CONNECTED = false;
```

Furthermore, the high-speed Direct Memory Access (DMA) for the ADC is hardcoded to only capture **two channels** (`ADC_CH_ENV` and `ADC_CH_RAW`):

```
adc_continuous_config_t adcCfg = {
  .pattern_num    = 2,       // Only capturing 2 channels!
```

Because the firmware is incapable of reading a third analog channel in its current state, and the Python offline analysis does not use REF data, repairing a broken `REF` pogo pin connection will have zero effect on your data. The `RAW` connection is the only critical fix.

Visual Proof (The "Ghost" Signal in Plotter)

If you run the captured logs through the offline plotter (`python sensordata_plotter.py --mode offline`), the visual output perfectly proves the disconnect.

You will observe:

1. **EMG RMS Flatline:** The calculated `EMG RMS` trace (blue) sits almost dead flat near zero, even while the hardware `ENV` trace (orange) shows massive spikes. If the RAW signal were connected, the RMS would trace the exact same mountain-peak shapes as the ENV line.
2. **Preprocessed EMG Flatline:** The `Preprocessed EMG` trace (blue) is a pure flatline.
3. **The Ghost RAW Trace:** The `Raw EMG` trace (black) shows very tiny, smoothed-out bumps that perfectly mimic the shape of the orange ENV line below it.

Why does RAW mirror ENV if it is disconnected? This is caused by **capacitive coupling**. A floating pin on a compact PCB acts like a tiny antenna. The copper trace for the `ENV` signal runs physically parallel to the floating `RAW` (A4) pin. When the muscle flexes, the strong, slow-moving DC voltage of the `ENV` signal induces a small, faint "ghost" voltage onto the nearby `RAW` trace.

Because this ghost signal is just a slow DC-like trace, when the Python script passes it through `preprocess_emg` (which applies a high-pass filter to block DC noise and keep only AC muscle frequencies), the ghost signal gets completely wiped out, resulting in the flatline seen in the preprocessed plot. Because the preprocessed signal is dead, all subsequent calculations (RMS, MDF, MNF) fail.

5. Verification

After applying the jumper wire, capture a new log. You should observe the following:

- **In the CSV:** At rest, the `raw` column should hover around `2048`. During a muscle flex, the `raw` column should oscillate rapidly and wildly up and down (e.g., swinging quickly between `500` and `3500`).
- **In the Plotter:** The `Raw EMG` and `Preprocessed EMG` traces will show dense, high-frequency "fuzz" (AC waveform) during flexes, and the `EMG RMS` line will spike up in perfect sync with the `ENV` line.